

# DNF-Lab: An Open Toolkit for Cognitive Architectures in Robotics

Weronika Wojtak\*  
*Centro de Computação Gráfica  
Research Centre Algoritmi  
Research Centre of Mathematics  
University of Minho  
Guimarães, Portugal  
weronika.wojtak@cgc.pt*

\*Corresponding author

Fabio Amadio  
*Inria HUCEBOT Team  
CNRS, LORIA  
Université de Lorraine  
Nancy, France  
fabio.amadio@inria.fr*

Emanuel Sousa  
*Centro de Computação Gráfica  
Guimarães, Portugal  
emanuel.sousa@cgc.pt*

Wolfram Erlhagen  
*Research Centre of Mathematics  
Department of Mathematics  
University of Minho  
Guimarães, Portugal  
wolfram.erlhagen@math.uminho.pt*

Estela Bicho  
*Research Centre Algoritmi  
Department of Industrial Electronics  
University of Minho  
Guimarães, Portugal  
estela.bicho@dei.uminho.pt*

**Abstract**—Dynamic Neural Fields (DNFs) provide a mathematically grounded framework for modeling cognitive processes in robotics, yet their adoption is hindered by the lack of accessible, reproducible implementations integrated with standard robotics workflows. Existing tools focus primarily on simulating neural field dynamics rather than demonstrating how DNFs interface with robotic perception, control, and middleware.

We present DNF-Lab, an open-source toolkit that bridges this gap by providing a complete, reproducible DNF-based cognitive architecture implemented in Python and ROS 2, fully integrated with CoppeliaSim for robotics simulation. The system demonstrates sequence learning and recall through two phases: a learning phase where human demonstrations are observed and encoded in memory fields, and a recall phase where a simulated Franka Emika arm autonomously reproduces the learned sequence with correct order and timing. The modular architecture, distributed as a Docker container with full source code on GitHub, provides researchers with a ready-to-run reference implementation that can be directly executed, inspected, and extended for cognitive robotics experiments.

By eliminating the need to rebuild core DNF infrastructure and demonstrating integration with standard robotics middleware, DNF-Lab lowers the barrier to experimenting with DNF-driven cognitive architectures in both simulated and physical robotic systems.

**Index Terms**—dynamic neural fields, cognitive robotics, ROS 2, experimental reproducibility

## I. INTRODUCTION

Translating cognitive architectures from theoretical models to functional robotic systems remains a significant challenge in cognitive robotics. While Dynamic Neural Fields (DNFs) have proven powerful for modeling cognitive processes such as memory, decision making, and sequential behavior [3], [8], their application in robotics is constrained by practical barriers.

Grounded in principles of cortical processing [1], [2] and widely used to bridge sensation and action in autonomous systems [3], DNFs provide both a design tool and a mathematical language for implementing higher-level cognitive functions. Over the past decades, DNF architectures have been used to endow robotic agents with capabilities including sequential task learning [8], value-based decision making [9], and temporally coordinated behavior [5]. However, most existing work focuses on the mathematical foundations and neural dynamics of DNFs, but provides limited support for integrating these models into robot control pipelines. Several software tools exist for simulating DNFs and exploring their dynamics [13], [14], and methodological work has addressed practical aspects such as parameter tuning [16], [15] and the analysis of localized activation states [4]. However, these resources are typically designed for studying the neural dynamics themselves rather than embedding DNFs within robot control workflows. While successful implementations of DNF architectures on physical robots have been reported in the literature [5]–[7], these demonstrations typically present architecture-specific results without providing the underlying implementation details, software infrastructure, or integration pipelines necessary for replication. The complete workflow connecting DNF models to robotic middleware, sensor processing, and actuator control often remains implicit, making it difficult for other researchers to adapt these approaches to their own robotic platforms. As a result, researchers working in robotics lack accessible, standardized examples of how to incorporate DNF architectures into realistic robotic systems and simulation environments.

To address this gap, the goal of this work is to intro-

duce DNF-Lab, an open and fully reproducible DNF-based cognitive architecture implemented in Python and ROS 2, a middleware framework that has become a de facto standard in robotics research and industrial applications [11], [12]. The framework is designed for seamless integration into robotics simulation environments and aims to make DNF-based approaches more accessible to robotics researchers. By providing a minimal yet functional architecture that can be run, inspected, and extended, DNF-Lab reduces the need to reimplement underlying computational mechanisms and enables experimentation with DNF-driven cognitive processes within workflows familiar to the robotics community.

The specific contributions of this work are as follows. (1) We introduce a foundational, open-source DNF-based cognitive architecture implemented in Python and ROS 2, fully integrated with CoppeliaSim to form a complete robotics workflow. The system demonstrates how DNFs can be embedded into standard ROS communication pipelines and provides a clear, extensible reference model for sequencing, memory, and timing in robotic tasks. (2) We provide a fully reproducible software environment, distributed as a ready-to-use Docker image containing all dependencies, along with open GitHub source code and documentation. A demonstrative scenario in which a human-performed action sequence is learned and later reproduced by a simulated Franka Emika arm illustrates how the framework can be directly executed, adapted, and extended for cognitive robotics research.

The remainder of the paper details the design of the DNF-Lab architecture, its integration within the ROS 2 and CoppeliaSim ecosystem, the experimental demonstration scenario, and potential avenues for future use and extension within cognitive robotics research.

## II. DYNAMIC NEURAL FIELDS

Dynamic Neural Fields (DNFs) provide a mathematically grounded framework for modeling the activation dynamics of recurrent neural populations [1]. Rather than tracking individual spikes, DNFs describe the evolution of averaged neural activity over continuous feature dimensions such as spatial location or movement direction [3]. Information is represented in the form of localized activity patterns, often referred to as bumps, that arise through the interaction of strong local excitation and broader inhibition [1]. Through these mechanisms DNFs generate self-sustained activation patterns that can remain stable even after the external input that triggered them has disappeared. These stable activation states function as attractors that encode continuous variables and support elementary cognitive capabilities such as working memory, decision making, and basic forms of prediction [3]. Much of the mathematical foundation underlying these models traces back to classical work by Wilson and Cowan [2] and by Amari [1]. In this work, we describe the field dynamics using the Amari neural field equation defined on a finite domain  $\Omega$ , which is chosen for the present application to be one-dimensional. While one-dimensional fields are sufficient for

representing the sequential structure considered here, higher-dimensional neural fields have been also used in prior work [9], [10]. The activity  $u(x, t)$  of the field on  $\Omega = [-L, L]$  then evolves according to

$$\tau \frac{\partial u(x, t)}{\partial t} = -u(x, t) + \int_{\Omega} w(x - y) f(u(y, t) - \theta) dy + I(x, t) + h. \quad (1)$$

Here,  $\tau$  is a time constant,  $I(x, t)$  denotes external input, and  $h$  sets the resting level of the field. The kernel  $w(x - y)$  specifies the pattern of lateral interactions, and  $f(u)$  is a firing rate function with threshold  $\theta$ .

This equation describes the dynamics of a basic neural field that serves as the foundation for more elaborate DNF architectures, which usually couple multiple fields to implement higher level cognitive processes.

DNFs are particularly well suited to cognitive robotics, where the central challenge is to integrate sensory information, maintain internal representations, and generate context-dependent actions [3]. Drawing on neurobiological processing principles, DNFs offer a way to embed cognitive functions directly into robot control architectures. Their ability to sustain localized activity patterns enables a robot to preserve goal-relevant information even once the corresponding sensory input disappears. Their competitive dynamics support robust selection and decision processes, and their continuous coupling to sensory signals allows rapid adaptation in dynamic environments. DNFs also naturally support anticipation by allowing internal representations to drive predictive or simulation-based processes, which is essential for tasks such as joint action, movement prediction, and adaptive timing. Together these properties position DNFs as a strong framework for bridging sensation and action in autonomous systems and for implementing cognitive capabilities that extend beyond reactive control [17].

Researchers interested in designing and studying neural field architectures benefit from several mature tools that support model construction, parameter exploration, and interactive experimentation. Two widely used frameworks are *Cedar* [13] and *Cosivina* [14]. *Cedar*, implemented in C++, enables large-scale simulations and facilitates integration with physical hardware through drag-and-drop model design. *Cosivina* is a Matlab-based framework that allows users to construct DNF architectures from modular elements, simulate their activation dynamics, and interactively adjust parameters via graphical user interfaces. These tools are complemented by resources that introduce the central concepts of DNF modeling, explain their grounding in neurophysiology and dynamical systems theory, and provide hands-on exercises and example architectures for exploring neural field dynamics [4], [17]. Additional methodological work has focused on practical parameter tuning, including studies that propose methods for setting suitable parameters [16] and on using numerical continuation to precisely determine ranges of parameters that support localized stable states [15]. Together, these resources

offer researchers a rich foundation for learning, experimenting with, and analyzing neural field models.

Despite the availability of extensive resources for learning and experimenting with DNFs, there remains a gap in accessible implementations that directly link these architectures to robotics workflows. Existing tools and frameworks primarily focus on understanding neural field dynamics or on simulating abstract architectures, but they do not readily integrate with robotics software ecosystems or provide complete examples connecting DNF models to robotic perception and control.

### III. SYSTEM OVERVIEW AND IMPLEMENTATION

#### A. DNF Architecture

To bridge this gap, we present a Dynamic Neural Field (DNF) implementation in Python and ROS 2 that is explicitly designed for integration into robotics workflows. The core of the system is a minimal DNF architecture capable of learning and recalling sequences with correct order and timing (Figure 1). While the underlying model has been described in prior work [8], it has been adapted here to interface directly with ROS 2 and CoppeliaSim, enabling end-to-end demonstrations that connect neural field dynamics to robotic perception and action.

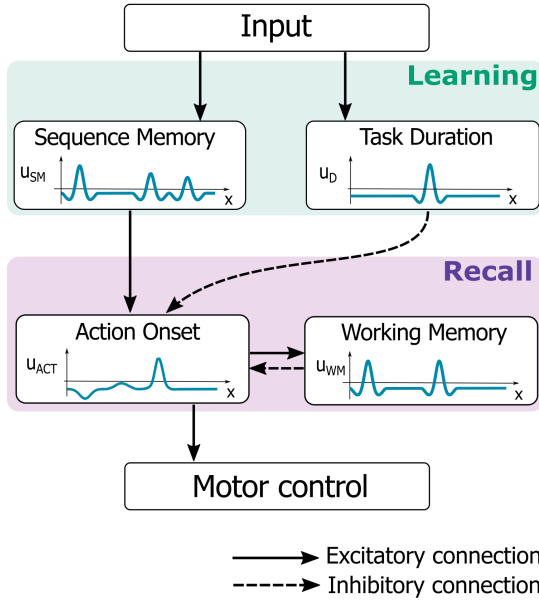


Fig. 1. Dynamic Neural Field (DNF) architecture for sequence learning and recall. During learning, memory and task duration fields are updated based on inputs; during recall, the architecture generates action signals for robot control. Solid lines indicate excitatory connections, while dashed lines indicate inhibitory connections.

The architecture comprises two functional components corresponding to the two phases of the experiment: a learning phase, in which sequence information is encoded from observation, and a recall phase, in which the learned sequence is autonomously reproduced. During the learning phase, two fields are active: the *sequence memory field*  $u_{sm}$  and the *task duration field*  $u_d$ . Gaussian inputs representing observed

human actions in the simulator are delivered to  $u_{sm}$ , triggering self-stabilized bumps that persist after the input has ceased. Threshold accommodation dynamics cause these bumps to grow over time, producing an activation gradient across the field, where the highest bump corresponds to the first event in the sequence [8]. This mechanism allows  $u_{sm}$  to encode the entire sequence, with activation strength decreasing according to the temporal order of events. The task duration field  $u_d$  receives a single input at the start of the task, producing a single bump whose growth over time provides a temporal reference for sequence recall.

During recall, the *action onset field*  $u_{act}$  and the *working memory field*  $u_{wm}$  are used to reproduce the learned sequence. The action onset field is preshaped using the memory gradient from  $u_{sm}$ , adjusted by the maximum of  $u_d$  to account for the initial delay. Sequence recall is initiated by gradually increasing the baseline activity of  $u_{act}$ , bringing neural populations closer to threshold and allowing self-stabilized bumps to emerge in temporal order. Activity in  $u_{act}$  drives  $u_{wm}$ , which maintains memory of already executed events, while inhibitory feedback suppresses recalled populations, enabling sequential progression. In the simulated experiment, this recall corresponds to the Franka Emika arm executing the learned sequence of actions based on the stored memory.

The specific implementation details, including the interaction kernels, firing rate functions, numerical parameter values, and equations for additional mechanisms (e.g., threshold accommodation dynamics), follow established formulations and are not reproduced here. These can be found in earlier work [3], [8] and can be examined directly in the shared code.

#### B. ROS 2 Nodes and Message Flow

The system uses a modular ROS 2 architecture to interface the DNF model with CoppeliaSim and the robot:

- **dnf\_system** package

- **cube\_input\_node**: Tracks cube positions in the simulator and publishes them on `/cuboid1_pos`, `/cuboid2_pos`, and `/cuboid3_pos`. These messages are converted into Gaussian inputs on `/dnf_inputs` for the learning node.
- **dnf\_learning\_node**: Implements the learning phase of the DNF architecture. It subscribes to `/dnf_inputs` to receive processed inputs representing observed human actions and updates the memory and task duration fields.
- **dnf\_recall\_node**: Implements the recall phase. It generates sequence predictions based on the learned memory and publishes them as strings on `/dnf_predictions`. This node operates feedforward with no feedback from the simulator.

- **coppelia\_bridge** package

- **coppelia\_bridge\_node**: Handles communication with CoppeliaSim. It streams cube positions to ROS topics, subscribes to `/dnf_predictions` to trigger robot commands, and sends these commands to the simulated Franka arm.

Figure 2 shows the high-level ROS 2 node architecture, illustrating the message flow between DNF nodes, the cube input node, and the CoppeliaSim bridge. Communication with CoppeliaSim uses the Remote API: during the learning phase, the `coppelia_bridge_node` connects to the simulator, retrieves cube handles, enables position streaming, and reads updated cube positions via `simxGetObjectPosition`. These values are published on `/cuboid1_pos`, `/cuboid2_pos`, and `/cuboid3_pos` and are converted into inputs on `/dnf_inputs`. During recall, the bridge node receives discrete predictions on `/dnf_predictions` and sends them to the simulator as string signals, which trigger the Franka arm to execute the learned pick-and-place sequence. Lua scripts inside CoppeliaSim control the human model (Bill) during learning and the robot arm during recall.

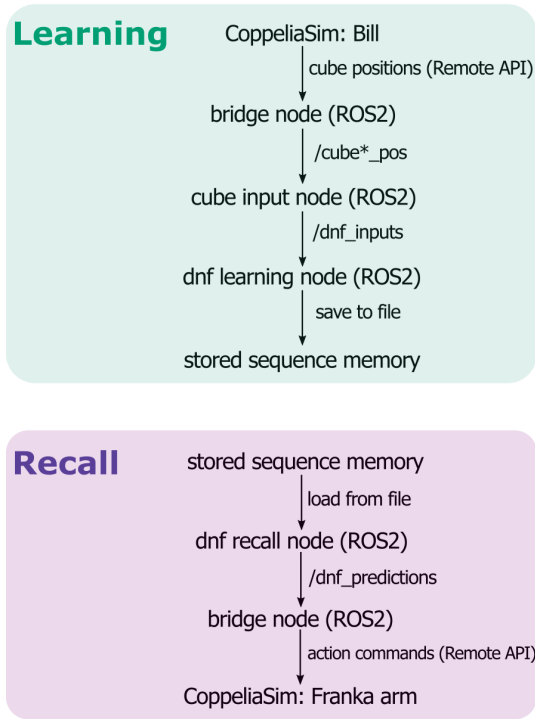


Fig. 2. High-level ROS 2 architecture for learning (top) and recall (bottom). During learning, cube positions are streamed from CoppeliaSim to the DNF system. During recall, predicted actions are sent from the DNF system through the bridge to CoppeliaSim to control the Franka arm.

The architecture supports two main experimental phases:

- 1) **Learning phase:** The cube input node publishes cube positions, which are converted to DNF inputs on `/dnf_inputs`. The `dnf_learning_node` updates the memory and task duration fields.
- 2) **Recall phase:** The `dnf_recall_node` publishes predicted actions to `/dnf_predictions`, which the bridge node translates into simulator commands that trigger robot motion through the embedded Lua control scripts.

This organization ensures modularity and a clear separation of concerns: perception and input processing are separate from DNF computation and robot control. Nodes can be launched via dedicated launch files (`dnf_learn.launch.py` and `dnf_recall.launch.py`) for each phase.

### C. Distribution and Reproducibility

The complete system, comprising DNF nodes, ROS 2 interface scripts, and the CoppeliaSim scenarios, is packaged in a Docker container to ensure reproducibility and simplify deployment. The Docker image includes all required Python packages, ROS 2 dependencies, and simulation components, and will be made publicly available on DockerHub ([hub.docker.com/r/wwojtak/coppelia-ros2-dnf](https://hub.docker.com/r/wwojtak/coppelia-ros2-dnf)). The source code and documentation, including instructions and usage examples, will be hosted on GitHub ([github.com/w-wojtak/coppelia-ros2-dnf](https://github.com/w-wojtak/coppelia-ros2-dnf)), enabling other researchers to reproduce experiments and adapt the system for their own studies.

All components of the system are freely available, facilitating accessibility. The implementation relies on Python and ROS 2, both open-source platforms, while the simulation is built in CoppeliaSim using the educational version, which is freely accessible. This setup allows others to replicate and extend the experiments without proprietary dependencies.

## IV. EXAMPLE SCENARIO IN COPPELIASIM

To demonstrate the DNF architecture in a robotic setting, we implemented a simplified pick-and-place scenario in CoppeliaSim involving three colored cubes on a table, a Franka Emika robot arm, and a virtual human demonstrator (Bill). Cube positions are continuously streamed to ROS via the `coppelia_bridge_node`, which converts these updates into Gaussian inputs for the DNF learning node, mimicking the sensory primitives a real robot might receive.

The experiment consists of two phases supported by separate scenes. In the learning phase, Bill executes pre-recorded trajectories that move the cubes sequentially. These movements provide time-varying input to the sequence memory field and the sequence-duration field, enabling the DNF to encode both the order and timing of the demonstrated sequence. In the recall phase, the learned dynamics are replayed: the DNF recall node generates predictions about the next action in the sequence, and the bridge node translates these predictions into commands for the Franka arm. The timing of the robot's actions is governed entirely by the internal DNF dynamics.

Figure 3 summarizes the full scenario. The panels in the left column (a, c, e) correspond to the learning phase. Panel (a) shows Bill manipulating the second cube, while the sequence memory field begins forming a bump at the position corresponding to cube 2. Panel (c) visualizes the activation patterns of the sequence memory field  $u_{sm}$  and the sequence-duration field  $u_d$  during learning; the localized input induced by Bill's action results in the formation of the second bump in  $u_{sm}$ , while  $u_d$  accumulates timing information. Panel (e) displays the temporal evolution of bump centers in  $u_{sm}$  over

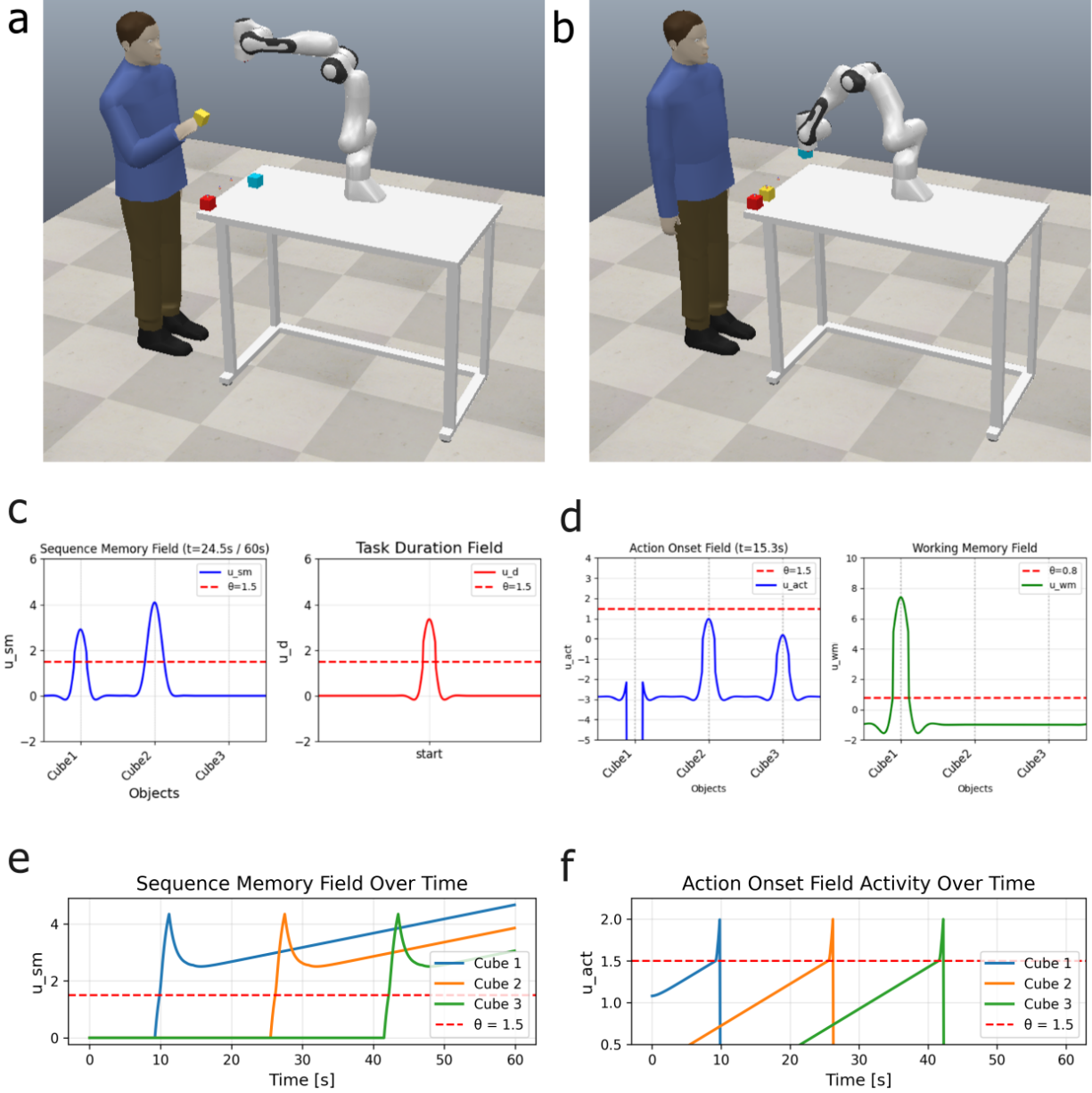


Fig. 3. Overview of the CoppeliaSim experiment demonstrating DNF-based sequence learning and recall. (a) Learning phase: the human demonstrator (Bill) moves the second cube, providing localized input to the DNF. (b) Recall phase: the robot executes the learned sequence; shown here is the first action, moving cube 1. (c) Activation of the sequence memory field  $u_{sm}$  and the sequence-duration field  $u_d$  during learning; input at the location of cube 2 leads to the formation of the second memory bump. (d) Activation of the action-onset field  $u_{act}$  and working-memory field  $u_{wm}$  during recall; threshold crossing in  $u_{act}$  at cube 1 triggers the corresponding robot action, while  $u_{wm}$  inhibits the executed location to enforce progression to cube 2. (e) Time course of bump centers in  $u_{sm}$  over the full 60-second learning trial. (f) Time course of bump centers in  $u_{act}$  during recall, illustrating that the temporal ordering of robot actions matches the demonstrated sequence.

the full 60-second learning trial, capturing the progression of the demonstrated sequence.

The right column (b, d, f) presents the corresponding processes during recall. Panel (b) shows the robot executing the first learned action by moving cube 1. Panel (d) depicts the activation of the action-onset field  $u_{act}$  and the working-memory field  $u_{wm}$ . The dominant bump in  $u_{act}$  has crossed threshold at the location corresponding to cube 1, triggering the robot action. Simultaneously, a bump in  $u_{wm}$  inhibits the

corresponding region in  $u_{act}$ , preventing re-selection of the completed action and enabling a transition to the next bump, corresponding to cube 2. Panel (f) shows the time course of bump centers in  $u_{act}$  across the 60-second recall phase, demonstrating that the robot's actions follow the order and timing structure encoded during learning.

This simulation setup provides a controlled, reproducible testbed in which DNF-based sequence learning and recall can be evaluated under realistic sensorimotor conditions before

deploying the architecture on a physical robot.

## V. DISCUSSION AND CONCLUSION

In this work, we presented DNF-Lab, a complete dynamic neural field architecture integrated with robotics simulation, demonstrating how DNF-based cognitive processes can be embedded within standard robotics workflows. The system successfully encoded and reproduced temporal sequences with correct order and timing, validating DNF-based sequence learning in a robotic context. The modular ROS 2 implementation allows researchers to extend the architecture for different cognitive functions and robotic platforms.

DNF-Lab addresses a critical gap between DNF theory and robotic practice. While frameworks such as *Cosivina* and *Cedar* excel at simulating neural field dynamics, they do not provide ready-to-use integration with robotics middleware. Our system complements these tools by demonstrating the complete pipeline from DNF models to robotic control, including sensor processing, message passing, and timing considerations that arise in practice. This approach reduces the effort required to move from theoretical architectures to functioning robotic implementations.

The current scenario focuses on sequential action learning as a fundamental building block for more complex behaviors. While the demonstration involves discrete pick-and-place actions in a controlled simulation, the underlying mechanisms of memory formation and temporal encoding generalize to broader task structures. The modular architecture supports natural extensions including sensory feedback, error monitoring, temporal adaptation, and multi-modal perception. Future work will implement these capabilities on physical platforms, completing the pathway from theory to deployed systems.

By providing open-source tools, documentation, and reproducible examples, DNF-Lab invites researchers to build upon this foundation—whether implementing alternative architectures, connecting to different platforms, or exploring novel applications. This work aims to accelerate the adoption of DNF-based cognitive architectures in robotics by lowering technical barriers and providing concrete reference implementations.

## ACKNOWLEDGMENT

This work is supported by the European Commission through a subgrant (SGA nr. euROBIN\_2OC\_5) funded by euROBIN, the European Robotics and AI Network (grant agreement 101070596). Views and opinions expressed are however those of the authors only and do not necessarily

reflect those of the European Union or European Commission. Neither the European Union nor the granting authority can be held responsible for them.

## REFERENCES

- [1] S. Amari, “Dynamics of pattern formation in lateral-inhibition type neural fields,” *Biological Cybernetics*, vol. 27, no. 2, pp. 77–87, 1977.
- [2] H. R. Wilson and J. D. Cowan, “A mathematical theory of the functional dynamics of cortical and thalamic nervous tissue,” *Kybernetik*, vol. 13, no. 2, pp. 55–80, 1973.
- [3] W. Erlhagen and E. Bicho, “The dynamic neural field approach to cognitive robotics,” *Journal of Neural Engineering*, vol. 3, no. 3, pp. R36, 2006.
- [4] S. Coombes, P. beim Graben, R. Potthast, and J. Wright, *Neural Fields: Theory and Applications*, Springer, 2014.
- [5] W. Wojtak, F. Ferreira, L. Louro, E. Bicho, and W. Erlhagen, “Adaptive timing in a dynamic field architecture for natural human–robot interactions,” *Cognitive Systems Research*, vol. 82, pp. 101148, 2023.
- [6] E. Bicho, W. Erlhagen, L. Louro, and E. C. Silva, “Neuro-cognitive mechanisms of decision making in joint action: A human–robot interaction study,” *Human Movement Science*, vol. 30, no. 5, pp. 846–868, 2011.
- [7] J. Tekülve, A. Fois, Y. Sandamirskaya, and G. Schöner, “Autonomous sequence generation for a neural dynamic robot: Scene perception, serial order, and object-oriented movement,” *Frontiers in Neurorobotics*, vol. 13, p. 95, 2019.
- [8] F. Ferreira, W. Wojtak, E. Sousa, L. Louro, E. Bicho, and W. Erlhagen, “Rapid learning of complex sequences with time constraints: A dynamic neural field model,” *IEEE Transactions on Cognitive and Developmental Systems*, vol. 13, no. 4, pp. 853–864, 2020.
- [9] W. Wojtak, F. Ferreira, P. Vicente, L. Louro, E. Bicho, and W. Erlhagen, “A neural integrator model for planning and value-based decision making of a robotics assistant,” *Neural Computing and Applications*, vol. 33, no. 8, pp. 3737–3756, 2021.
- [10] W. Wojtak, S. Coombes, D. Avitabile, E. Bicho, and W. Erlhagen, “Robust working memory in a two-dimensional continuous attractor network,” *Cognitive Neurodynamics*, vol. 18, no. 6, pp. 3273–3289, 2024.
- [11] F. A. H. Fernando, A. H. Arvin, et al., “Load pushing capacity analysis of individual and multi-cooperative mobile robot through symbiotic application,” *International Journal of Mechanical Engineering and Robotics Research*, vol. 13, pp. 304–313, 2024.
- [12] D. Krūmiņš, et al., “Open-source web lab for remote and on-site robotics practice in a realistic zero-setup ROS 2 development environment,” *IEEE Robotics and Automation Practice*, 2025.
- [13] O. Lomp, M. Richter, S. K. U. Zibner, and G. Schöner, “Developing dynamic field theory architectures for embodied cognitive systems with cedar,” *Frontiers in Neurorobotics*, vol. 10, p. 14, 2016.
- [14] S. Schneegans, “COSIVINA: A Matlab toolbox to compose, simulate, and visualize neurodynamic architectures,” 2021.
- [15] J. Rankin, D. Avitabile, J. Baladron, G. Faye, and D. J. B. Lloyd, “Continuation of localized coherent structures in nonlocal neural field equations,” *SIAM Journal on Scientific Computing*, vol. 36, no. 1, pp. B70–B93, 2014.
- [16] F. Alexandre, J. Fix, A. Hutt, N. P. Rougier, and T. Viéville, “On practical neural field parameters adjustment,” in *Deuxième conférence française de Neurosciences Computationnelles, Neurocomp08*, 2008.
- [17] G. Schöner, J. Spencer, DFT Research Group, et al., *Dynamic Thinking: A Primer on Dynamic Field Theory*, Oxford: Oxford University Press, 2015.